

Chapitre 4:

SQL :

**Le Langage d'interrogation
des bases de données (LID)**

SQL - LID

- Projection
- Sélection
- Produit cartésien
- jointure
- Fonction de groupe
- Opération ensembliste
- Sous interrogation
- Extraction hiérarchique

Interroger les BDDs relationnelles?

- La principale commande du langage d'interrogation de données est la commande SELECT.
- La commande SELECT est basée sur l'algèbre relationnelle, en effectuant des opérations de sélection de données sur plusieurs tables relationnelles par projection.

Sa syntaxe générale est la suivante :

```
SELECT [ALL] | [DISTINCT] <liste des attributs> | *  
FROM <liste des tables>  
[WHERE <condition> ]  
[GROUP BY <liste des attributs> ]  
[HAVING <condition de groupement>]  
[ORDER BY <liste des attributs de tri>] ;
```



Interroger les BDDs relationnelles?

Une seule instruction: **SELECT**

Un ordre SELECT permet d'extraire des informations d'une base de données. L'utilisation d'un ordre SELECT offre les possibilités suivantes :

- Sélection : SQL permet de choisir dans une table, les lignes que l'on souhaite ramener au moyen d'une requête.
- Projection : SQL permet de choisir dans une table, les colonnes que l'on souhaite ramener au moyen d'une requête.
- Jointure : SQL permet de joindre des données stockées dans différentes tables, en créant un lien par le biais d'une colonne commune à chacune des tables.

Interroger les BDDs relationnelles?

Soit la BDD composée des relations suivantes :

- **Produit** (Nump, nomp, categorie, prix)
- **Depot** (Numdep, libellé, adr-dep)
- **Stock** (Nump, Numdep, qte)

La projection

Une projection est une instruction permettant de sélectionner un ensemble d'attributs dans une table.

Syntaxe :

SELECT [ALL/DISTINCT] < expression de valeurs>
FROM nom de relation

<Expression de valeurs>: expression arithmétique composée :

- d'opérateurs binaires (+, -, *, /)
- de constantes
- de colonnes.

N.B: Lors de l'exécution de l'instruction SELECT, MySQL évalue la clause FROM avant la clause SELECT

Projection: **exemples**

Q1) donner la liste des numéros de produits

Q2) donner les Catégories de produits (possibilité d'homonymie)

Projection: **exemples**

SELECT Nump
FROM Produit

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50



Nump
1
2
3
4
5

La projection

Éliminer les redoublons: La commande **DISTINCT** permet de récupérer des valeurs uniques dans une ou plusieurs colonnes. Elle élimine les doublons dans les résultats d'une requête.

Exemples: Donner les catégories de produits

Select categorie
from produit



categorie
Papeterie
Papeterie
Informatique
Informatique
Informatique

Projection: **exemples**

SELECT **DISTINCT** categorie
FROM Produit

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50



categorie
Papeterie
Informatique

La projection

Obtenir plusieurs colonnes: Avec la même table, il est possible de lire plusieurs colonnes à la fois. Il suffit de séparer les noms des champs souhaités par une virgule.

Exemples: Récupérer les noms des produits et leurs catégories

Select nomp, categorie
from produit



nomp	categorie
Stylo	Papeterie
Cahier	Papeterie
Clavier	Informatique
Souris	Informatique
USB	Informatique

Projection: exemples

Exemples: Récupérer les noms des produits ainsi que leurs prix en ajoutant 10 DH à chaque prix

Select nomp, prix+10
from produit



nomp	Prix+10
Stylo	15.00
Cahier	22.50
Clavier	160.00
Souris	90.00
USB	65.50

Exprimer la sélection ?

La sélection (ou **filtrage**) permet de récupérer des données d'une ou plusieurs tables en fonction de critères spécifiques. Elle est effectuée avec la commande **SELECT** et la clause **WHERE**.

Syntaxe :

SELECT *

FROM nom-table

WHERE qualification (condition de recherche)

- Une condition de recherche élémentaire est appelée prédicat en SQL.
- Un prédicat compare deux expressions de valeurs ; la première est appelée terme (contenant des colonnes) et la seconde constante.

Prédicats en SQL

Prédicat de comparaison : Les opérateurs { =, !=, <, >, >=, <= }

Prédicat d'intervalle : BETWEEN

Prédicat de comparaison de texte : LIKE

Prédicat de test de nullité : IS NULL

Prédicat d'appartenance : IN

La sélection

Il est possible de retourner automatiquement toutes les colonnes d'un tableau sans avoir à connaître le noms de toutes les colonnes. Au lieu de lister toutes les colonnes, il faut utiliser le caractère *

Exemple: **Select** *

From produit

Where categorie= 'Informatique';



Nump	nomp	categorie	prix
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

Alias ou synonyme de relations ?

Un alias : Un alias est un nom temporaire attribué à une relation ou une colonne afin de simplifier l'écriture des requêtes SQL et d'améliorer la lisibilité des résultats.

Un alias peut être utilisé :

- ✓ Dans la clause SELECT pour renommer une colonne dans l'affichage des résultats.
- ✓ Dans la clause FROM pour attribuer un nom plus court à une table.

3 formes pour désigner les attributs d'une relation

- Nom-attribut, si non ambiguïté
- Nom-relation.nom-attribut si ambiguïté
- Alias-relation.nom-attribut si ambiguïté.

Alias ou synonyme de relations ?

Syntaxe:

SELECT attr1 AS **aliasa1**, attr2 AS **aliasa2**, ...

FROM table1 **aliast1**, table2 **aliast2**... ;

Pour les attributs, l'alias correspond aux titres des colonnes affichées dans le résultat de la requête.

Il est souvent utilisé lorsqu'il s'agit d'attributs calculés(expression).

NB : Le mot clé AS est optionnel

Alias ou synonyme de relations

Exemples d'utilisation des alias

Alias dans Select:

SELECT nomp, prix * 1.1 **AS** Nouveau_prix
FROM produit;



nomp	Nouveau_prix
Clavier	165.00
Souris	88.00
USB	61.05

nomp	prix
Stylo	5.00
Cahier	12.50
Clavier	150.00
Souris	80.00
USB	55.50

Alias dans From:

SELECT p.nomp, p.prix
FROM produits **AS** p;



La restriction

Une restriction consiste à sélectionner les lignes satisfaisant à une condition logique effectuée sur leurs attributs.

En SQL, les restrictions s'expriment à l'aide de la clause **WHERE** suivie d'une condition logique.

La commande WHERE dans une requête SQL permet d'extraire des lignes d'une base de données qui respectent une condition. Cela permet d'obtenir uniquement les informations désirées.

Syntaxe:

```
Select Nom_Champ  
From Nom_Table  
Where Condition
```

La Commande Where

Opérateurs: Ils existent plusieurs opérateurs à utilisés avec la clause WHERE. La liste ci-jointe présente les opérateurs les plus couramment utilises.

Opérateurs logiques:

- AND
- OR
- NOT

Comparateur logique:

- IN
- BETWEEN
- LIKE

Opérateurs Arithmétiques:

- +
- -
- *
- /
- %

Comparateurs Arithmétiques:

- =
- !=
- <
- >
- <=
- >=
-

Commande Where

Opérateurs logiques AND et OR:

Une requête SQL peut être restreinte à l'aide de la condition WHERE. Les opérateurs logiques **AND** et **OR** peuvent être utilisées au sein de la commande WHERE pour combiner des conditions.

Les opérateurs sont à ajouter dans la condition WHERE. Ils peuvent être combinés à l'infini pour filtrer les données comme souhaités.

L'opérateur **AND** permet de s'assurer que la **condition1** **ET** la **condition2** sont vraies

L'opérateur **OR** vérifie quant à lui que la **condition1** **OU** la **condition2** est vraie

```
Select Nom_Champ  
From Nom_Table  
Where Condition1 AND  
Condition2
```

```
Select Nom_Champ  
From Nom_Table  
Where Condition1 OR  
Condition2
```

Ces opérateurs peuvent être combinés à l'infini

```
Select Nom_Champ From Nom_Table  
Where Condition1 AND (Condition2 OR Condition3)
```

Exprimer la sélection: Exemples

Récupérer la liste des produits de la catégorie "Informatique" dont le prix est supérieur à 70.

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

```

SELECT      *
FROM        Produit
WHERE       categorie='informatique' and prix>70;
  
```

Nump	nomp	categorie	prix
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00

Exprimer la sélection: Exemples

Récupérer la liste des produits dont le nom est "clavier" ou "USB"

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

```

SELECT      *
FROM        Produit
WHERE       nomp='Clavier' OR nomp='USB';
  
```

Nump	nomp	categorie	prix
3	Clavier	Informatique	150.00
5	USB	Informatique	55.50

Exprimer la sélection: Exemples

Récupérer tous les produits appartenant à la catégorie "informatique" avec un prix supérieur à 70 ou à la catégorie "Papeterie" avec un prix inférieur à 10.

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

```
SELECT *
FROM Produit
WHERE (categorie='informatique' and prix>70) OR (categorie='Papeterie' and
prix<10) ;
```

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00

Commande Where

L'opérateur IN:

L'opérateur logique IN dans SQL s'utilise avec la commande WHERE pour vérifier si une colonne est égale à une des valeurs comprise dans un ensemble déterminé de valeurs.

C'est une méthode simple pour vérifier si une colonne est égale à une valeur **OU** une autre valeur **OU** une autre valeur **OU** une autre valeur et ainsi de suite, sans avoir à utiliser de multiple fois l'opérateur **OR**.

```
Select Nom_Champ  
From Nom_Table  
Where Nom_champ IN (valeur1,valeur2,valeur3....)
```

NB: entre les parenthèses il n'y a pas de limite du nombre d'arguments. Il est possible d'ajouter encore d'autres valeurs.

Cette syntaxe peut être associée à l'opérateur **NOT** pour rechercher toutes les lignes qui ne sont pas égales à l'une des valeurs stipulées.

Exprimer la sélection: Exemple

Récupérer les numéros des produits dont le nom est "clavier", "cahier" ou "stylo"

❑ Requête avec plusieurs OR

SELECT Nump

FROM Produit

WHERE nomp = 'clavier' OR nomp = 'cahier' OR nomp = 'stylo';

❑ Requête equivalent avec l'opérateur IN

SELECT Nump

FROM Produit

WHERE nomp **IN** ('clavier', 'cahier', 'stylo');

Exprimer la sélection: Exemple

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

```

SELECT    Nump
FROM      Produit
WHERE     nomp IN ('stylo', 'cahier', 'clavier');
  
```

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00

Commande Where

L'opérateur BETWEEN:

L'opérateur BETWEEN est utilisé dans une requête SQL pour sélectionner un intervalle de données dans une requête utilisant WHERE. L'intervalle peut être constitué de chaînes de caractères, de nombres ou de dates.

```
Select *  
From Nom_Table  
Where Nom_colonne BETWEEN 'valeur1' AND 'valeur2'
```

La requête suivante retournera toutes les lignes dont la valeur de la colonne « nom_colonne » est comprise entre valeur 1 Et valeur 2 .

Exprimer la sélection: Exemples

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

- La liste des numéros de produits dont le prix est compris entre 100 et 300 DH

```

SELECT  Nump
FROM    Produit
WHERE   prix BETWEEN 100 AND 300;

```

Nump
3

Exprimer la sélection: Exemples

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

- La liste des produits dont le numéro produit n'est pas compris entre 3 et 5

```

SELECT *
FROM   Produit
WHERE  Nump Not BETWEEN 3      AND 5;

```

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50

Commande Where

L'opérateur LIKE:

L'opérateur LIKE est utilisé dans la clause WHERE des requêtes SQL. Ce mot-clé permet d'effectuer une recherche sur un modèle particulier. Il est par exemple possible de rechercher les enregistrements dont la valeur d'une colonne commence par telle ou telle lettre:

```
Select *  
From Nom_Table  
Where Nom_colonne LIKE modèle
```

Un modèle permet de définir un critère de filtrage des données selon sa structure.

Commande Where

L'opérateur LIKE: Un modèle de recherche peut être défini avec l'opérateur LIKE, qui permet d'effectuer des correspondances partielles sur des chaînes de caractères. Voici quelques exemples courants :

- **LIKE '%a':** Le caractère % est un joker qui remplace tous les autres caractères. Ce modèle permet donc de rechercher toutes les chaînes qui se terminent par la lettre « a ».
- **LIKE 'a%':** Ce modèle permet de rechercher toutes les valeurs qui commencent par la lettre « a ».
- **LIKE '%a%':** Ce modèle est utilisé pour rechercher toutes les valeurs contenant la lettre « a » à n'importe quelle position.
- **LIKE 'pa%on':** Ce modèle permet de rechercher les chaînes qui commencent par « pa » et se terminent par « on », comme « pantalon » ou « pardon ».
- **LIKE 'a_c':** Le caractère _ (underscore) peut être remplacé par **un seul caractère quelconque**. Ainsi, ce modèle permet de retourner les valeurs comme « aac », « abc » ou encore « azc ».

Exprimer la sélection: Exemples

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

- La liste des numéros de produits dont le nom commence par la lettre 'C' ?

```

SELECT  Nump
FROM    Produit
WHERE   nomp LIKE 'C%';

```

Nump
2
3

Exprimer la sélection: Exemples

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00
2	Cahier	Papeterie	12.50
3	Clavier	Informatique	150.00
4	Souris	Informatique	80.00
5	USB	Informatique	55.50

- La liste des produits dont le nom se termine par « O »?

```

SELECT *
FROM Produit
WHERE nomp LIKE '%O';

```

Nump	nomp	categorie	prix
1	Stylo	Papeterie	5.00

Commande Where

L'opérateur IS NULL / IS NOT NULL:

Dans le langage SQL, l'opérateur IS permet de filtrer les résultats qui contiennent la valeur NULL. Cet opérateur est indispensable car la valeur NULL est une valeur inconnue et ne peut par conséquent pas être filtrée par les opérateurs de comparaison(égal, inférieur, supérieur ou différent).

```
Select *  
From Nom_Table  
Where Nom_colonne IS NULL
```

A l'inverse , pour filtrer les résultats et obtenir uniquement les enregistrements qui ne sont pas nuls

```
Select *  
From Nom_Table  
Where Nom_colonne IS NOT NULL
```

Exemples

Nump	Numdep	Qte
1	10	50
2	20	NULL
3	30	20
4	40	65
5	50	NULL

- Afficher les articles en stock dont la quantité n'est pas renseignée

```
SELECT *  
FROM stock  
WHERE qte IS NULL;
```

Nump	Numdep	Qte
2	20	NULL
5	50	NULL

Exemples

Nump	Numdep	Qte
1	10	50
2	20	NULL
3	30	20
4	40	65
5	50	NULL

- Afficher les articles en stock avec une quantité connue

```
SELECT *  
FROM stock  
WHERE qte IS NOT NULL;
```

Nump	Numdep	Qte
1	10	50
3	30	20
4	40	65

Exercice

Soit le schéma de la BDD :

- **Employé**(codemp, nomemp, datrecrut, codposte, salaire, prime, supérieur, coddep)
- **Département**(coddep, nomdep, codrespon)
- **Poste**(codposte, intitulé, salmin, salmax)
- **Historique-poste**(codemp, codposte, coddep, datdebut, datfin)

Exercice

1. Affiche les employés qui ont un salaire supérieur à 3000 et qui sont dans le département dont le code est 10.
2. Affiche le nom des employés dont le nom commence par "A" et qui ont été recrutés après le 1er janvier 2018.
3. Affiche les départements qui n'ont pas de responsable
4. Affiche les employés qui travaillent dans les départements dont les codes sont 5, 8 ou 12.
5. Affiche les employés dont le salaire est compris entre 3000 et 6000, et qui ont été recrutés entre le 1er janvier 2015 et le 31 décembre 2020.
6. Affiche les employés dont le salaire est supérieur à 4000 et qui sont dans les départements 3 ou 5, ou bien qui ont un supérieur avec le code 100.

Clause ORDER BY

ORDER BY précise l'ordre dans lequel la liste des lignes sélectionnées dans le résultat sera affichée.

Syntaxe : SELECT colonne1, colonne2
FROM table
ORDER BY colonne1, colonne2 [DESC];

Par défaut, l'ordre est croissant. **DESC** : tri par ordre décroissant.
Le tri se fait d'abord selon la première expression, puis les lignes ayant la même valeur pour la première expression sont triées selon la deuxième, etc.

Les valeurs nulles sont toujours en tête quel que soit l'ordre du tri

SELECT colonne1, colonne2
FROM table
ORDER BY colonne1 DESC, colonne2 ASC;

ORDER BY : Exemples

Exemple 1:

Liste des employés classé par ordre alphabétique des noms

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P002	4500	400	104	D20
102	Ali	2019-06-12	P001	2000	600	102	D10
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20

ORDER BY : Exemples

Select *

From employé

Order by nomemp;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
102	Ali	2019-06-12	P001	2000	600	102	D10
101	karim	2020-09-25	P002	4500	400	104	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20

ORDER BY : Exemples

Exemple 2:

Liste des noms d'employés et de leur poste, triée par département et dans chaque département par ordre de salaire décroissant

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P002	4500	400	104	D20
102	Ali	2019-06-12	P001	2000	600	102	D10
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20

ORDER BY : Exemples

SELECT nomemp, codposte
FROM employe
ORDER BY coddep , salaire **DESC**;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
102	Ali	2019-06-12	P001	2000	600	102	D10
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20
104	Yasmine	2020-04-18	P004	5200	200	102	D20
101	karim	2020-09-25	P002	4500	400	104	D20
103	Sarah	2018-11-03	P003	3800	350	102	D30

Fonctions intégrées

MySQL, Comme les autres systèmes de gestion de bases de données, propose de nombreuses fonctions intégrées qui permettent de manipuler les données utilisateurs ou les données du système.

Il existe plusieurs catégories de fonctions :

- Fonctions mathématiques
- Fonctions de traitement de chaînes
- Fonctions de manipulation de dates
- Fonctions de conversion

Fonctions mathématiques

Ces fonctions prennent un ou plusieurs nombres en arguments et renvoient une valeur numérique.

Voici quelques exemples :

Fonction	Description
ABS()	Retourne la valeur absolue d'un nombre.
CEIL()	Renvoie la plus petite valeur entière supérieure ou égale au nombre d'entrée.
FLOOR()	Renvoie la plus grande valeur entière inférieure ou égale au nombre d'entrée.
MOD()	Renvoie le reste d'un nombre divisé par un autre
ROUND()	Arrondit un nombre à un nombre spécifié de places décimales.
TRUNCATE()	Tronque un nombre à un nombre spécifié de places décimales.

Fonctions mathématiques: Exemples

Fonction	Résultat
ABS(-15)	15
CEIL(4.2)	5
CEIL(-4.8)	-4
FLOOR(4.8)	4
FLOOR(-4.2)	-5
MOD(17, 5)	2
ROUND(3.14159, 2)	3.14
ROUND(27.376, 1)	27.4
ROUND(27.376)	27
Truncate(27.376, 1)	27.3
Truncate(27.376, 0)	27

Fonctions mathématiques: Exemples

Afficher le nom de chaque employé ainsi que son salaire arrondi de deux chiffres après la virgule.

```
Select nomemp, round(salaire,2)
```

```
From employe;
```

nomemp	Salaire
karim	4500.00
Ali	2000.00
Sarah	3800.00
Yasmine	5200.00
Omar	5900.00

Fonctions mathématiques: Exemples

Pour chaque employé, affichez le matricule, le nom, le salaire et le salaire augmenté de 15% sous la forme d'un nombre entier. Nommez cette colonne New Salary.

Select codemp,nomemp, salaire, round(salaire*1.15) As "New Salary"

From employe;

codemp	nomemp	Salaire	New Salary
101	karim	4500	5175
102	Ali	2000	2300
103	Sarah	3800	4370
104	Yasmine	5200	5980
105	Omar	5900	6785

Fonctions de traitement de chaînes

Les fonctions de chaîne les plus couramment utilisées permettent de manipuler efficacement les données textuelles. Voici une liste de ces fonctions:

Name	Description
CONCAT	Concaténer deux ou plusieurs chaînes en une seule chaîne
INSTR	Renvoie la position de la première occurrence d'une sous-chaîne dans une chaîne
LENGTH	Obtenir la longueur d'une chaîne en octets et en caractères
LEFT	Obtient un nombre spécifié de caractères les plus à gauche d'une chaîne
LOWER	Convertir une chaîne en minuscule
TRIM	Supprime les caractères indésirables d'une chaîne
LTRIM	Supprimer tous les espaces du début d'une chaîne
RTRIM	Supprime tous les espaces de la fin d'une chaîne

Fonctions de traitement de chaînes

Name	Description
REPLACE	Recherche et remplace une sous-chaîne dans une chaîne
RIGHT	retourne un nombre spécifié de caractères les plus à droite d'une chaîne
SUBSTRING	Extraire une sous-chaîne à partir d'une position avec une longueur spécifique.
SUBSTRING_INDEX	Renvoie une sous-chaîne à partir d'une chaîne avant un nombre spécifié d'occurrences d'un délimiteur
LPAD	Compléter une chaîne de caractère jusqu'à ce qu'elle atteigne la taille souhaitée
FIND_IN_SET	Rechercher une chaîne dans une liste de chaînes séparées par des virgules
FORMAT	Mettre en forme un nombre avec une locale spécifique, arrondi au nombre de décimales
UPPER	Convertir une chaîne en majuscule

Fonctions de traitement de chaînes: Exemples

Name	Résultat
CONCAT('Une', 'Maison')	UneMaison
INSTR('Database', 'base')	5(position du mot "base")
LENGTH('MySQL')	5(nombre de caractères)
UPPER('Cours Sql')	COURS SQL
LOWER('MYSQL')	mysql(converti en minuscules)
LPAD('123', 6, '\$')	\$\$\$123
REPLACE('JACK and JUE', 'J', 'BL')	BLACK and BLUE
TRIM('H' from 'Helloworld')	elloworld
SUBSTRING('Hello World!', 1, 5)	Hello

Fonctions de traitement de chaînes: Exemple

Afficher le numéro de l'employé ainsi qu'une colonne "Information" qui s'affiche sous la forme suivante : **<nom employé> gagne <salaire> par mois**, puis remplacer le préfixe "D" dans le code du département par "**dep**".

```
SELECT codemp,
CONCAT(nomemp, ' gagne ', salaire, ' par mois') AS Information,
REPLACE(coddep, 'D', 'Dep')
FROM employe;
```

Codemp	Information	Coddep
101	Karim gagne 4500 par mois	Dep20
102	Ali gagne 2000 par mois	Dep10
103	Sarah gagne 3800 par mois	Dep30
104	Yasmine gagne 5200 par mois	Dep20
105	Omar gagne 5900 par mois	Dep20

Fonctions de manipulation de dates

Les fonctions de manipulation de dates les plus couramment utilisées en MySQL permettent de traiter efficacement les données de type date. Voici une liste de ces fonctions.

Name	Description
CURDATE	Renvoie la date actuelle.
DATEDIFF	Calcule le nombre de jours entre deux valeurs DATE.
DAY	Obtient le jour du mois d'une date spécifiée.
DATE_ADD	Ajoute une valeur de temps à la valeur de date.
DATE_SUB	Soustrait une valeur d'heure d'une valeur de date.
DATE_FORMAT	Formate une valeur de date en fonction d'un format de date spécifié.
DAYNAME	Obtient le nom d'un jour de la semaine pour une date spécifiée.
DAYOFWEEK	Renvoie l'index des jours de la semaine pour une date.

Fonctions de manipulation de dates

Name	Description
EXTRACT	Extrait une partie d'une date.
LAST_DAY	Renvoie le dernier jour du mois d'une date spécifiée
NOW	Renvoie la date et l'heure actuelles d'exécution de l'instruction.
MONTH	Renvoie un entier qui représente un mois d'une date spécifiée.
STR_TO_DATE	Convertit une chaîne en une valeur de date et d'heure basée sur un format spécifié.
SYSDATE	Renvoie la date actuelle.
TIMEDIFF	Calcule la différence entre deux valeurs TIME ou DATETIME.
TIMESTAMPDIFF	Calcule la différence entre deux valeurs DATE ou DATETIME.
WEEK	Renvoie le numéro de semaine d'une date
WEEKDAY	Renvoie un index des jours de la semaine pour une date.
YEAR	Renvoie l'année pour une date spécifiée

Fonctions de manipulation de dates: Exemple

Afficher la liste des employés(codemp et nomemp), leur **date de recrutement**, **l'année de recrutement**, **le jour de la semaine** correspondant à leur date de recrutement et le **nombre de jours** écoulés depuis leur recrutement jusqu'au **aujourd'hui**.

```
SELECT
    codemp,
    nomemp,
    datrecrut,
    YEAR(datrecrut) AS Année_Recrutement,
    DAYNAME(datrecrut) AS Jour_Semaine,
    DATEDIFF(CURDATE(), datrecrut) AS Jours_ecoules
FROM employe;
```


Fonctions de manipulation de dates: Exemple

Résultat:

codemp	nomemp	datrecrut	annee_recruitment	jour_semaine	jours_ecoules
101	karim	2020-09-25	2020	vendredi	1631
102	Ali	2019-06-12	2019	mercredi	2126
103	Sarah	2018-11-03	2018	samedi	2316
104	Yasmine	2020-04-18	2020	samedi	1792
105	Omar	2017-07-30	2017	dimanche	2702

Fonctions de conversion

Dans MySQL, les fonctions de conversion permettent de transformer les valeurs d'un type de données à un autre. Voici les principales fonctions utilisées pour la conversion :

DATE_FORMAT() – Conversion de date en texte.

STR_TO_DATE() – Conversion de texte en date.

CAST() et **CONVERT()** – Conversion de types numériques et texte.

Fonctions de conversion

Conversion des Dates avec DATE_FORMAT()

Syntaxe :

DATE_FORMAT(date, 'format')

Exemple :

```
SELECT DATE_FORMAT('2024-03-23', '%d/%m/%Y') AS  
date_formatee;
```

Résultat : 23/03/2024

Fonctions de conversion

Formats disponibles :

Format	Signification	Exemple
%Y	Année sur 4 chiffres	2024
%y	Année sur 2 chiffres	24
%M	Mois en texte	March
%m	Mois en chiffres	03
%d	Jour du mois	23
%W	Jour de la semaine	Saturday
%H	Heure (24h)	15
%h	Heure (12h)	03
%i	Minutes	45
%s	Secondes	30

Fonctions de conversion

Ecrire une requête pour extraire le code, le nom, l'année et le mois de la date de recrutement de chaque employé, et de les afficher sous le format "YYYY-MM". La colonne résultante est nommée `annee_mois_recrut`.

```
SELECT codemp, nomemp, DATE_FORMAT(datrecurt, '%Y-%m') AS  
annee_mois_recrut  
FROM employe;
```

codemp	nomemp	annee_mois_recrut
101	Karim	2020-09
102	Ali	2019-06
103	Sarah	2018-11
104	Yasmine	2020-04
105	Omar	2017-07

Fonctions de conversion

Conversion d'une Chaîne en Date avec STR_TO_DATE()

Syntaxe :

`STR_TO_DATE('chaîne', 'format')`

Exemple :

`STR_TO_DATE('23 mars 2024', '%d %M %Y')`

Résultat : 2024-03-23

Fonctions de conversion

Conversion des Types avec CAST() et CONVERT()

Ces fonctions permettent de convertir des valeurs en différents types de données.

- **Conversion en Texte**

Syntaxe :

CAST(expression AS CHAR) ou **CONVERT(expression, CHAR)**

- **Conversion en Nombre**

Syntaxe :

CAST(expression AS DECIMAL) ou **CONVERT(expression, DECIMAL)**

Exemple : obtenir un salaire avec des décimales

```
SELECT codemp, nomemp, CAST(Salaire AS DECIMAL(10, 2)) AS  
salaire_decimal  
FROM employe;
```

Fonctions de conversion

Résultat:

codemp	nomemp	salaire_decimal
101	Karim	4500.00
102	Ali	2000.00
103	Sarah	3800.00
104	Yasmine	5200.00
105	Omar	5900.00

Fonctions de conversion

Conversion et gestion des valeurs NULL

→ Qu'est-ce qu'une valeur NULL ?

En SQL, une valeur **NULL** représente une **valeur inconnue ou manquante** dans une base de données.

→ Problème avec les valeurs NULL :

Les opérations sur des valeurs NULL (par exemple, une addition ou une comparaison) renvoient également NULL, ce qui peut poser problème dans des calculs ou lors de l'affichage des résultats.

Fonctions de conversion

Conversion et gestion des valeurs NULL

1. IFNULL(): Permet de remplacer une valeur NULL par une valeur de remplacement.

Syntaxe : IFNULL(expression, valeur_de_replacement)

2. COALESCE(): Retourne la première valeur non-NULL parmi plusieurs expressions.

Syntaxe : COALESCE(expression1, expression2, ..., expressionN)

Exemples:

1- SELECT nomemp, IFNULL(Salaire, 0) AS salaire_non_null
FROM employe;

2- SELECT nomemp, COALESCE(Salaire, Prime, 0) AS salaire_ou_prime
FROM employe;

Exprimer le produit cartésien ?

- le produit cartésien s'exprime comme une jointure sans qualification :

Syntaxe :

SELECT *
FROM liste de noms de relations

Exemples :

- La requête en algèbre relationnelle : $P := \text{produit} \times \text{stock}$ s'exprime en SQL comme suit :

SELECT *
FROM Produit, Stock;

Produit cartésien (normeSQL2)

- La norme SQL2 a aussi une syntaxe spéciale pour le produit cartésien de deux tables :
- Exemple:
SELECT Nomemp, nomdep
FROM employe **CROSS JOIN** departement

Produit cartésien (normeSQL2)

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P002	4500	400	104	D20
102	Ali	2019-06-12	P001	6000	600	102	D10
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20

coddep	Nomdep	codrespon
D10	Informatique	102
D20	RH	104
D30	Finance	103
D40	Marketing	NULL

Produit cartésien (normeSQL2)

SELECT Nomemp, nomdep
FROM employe **CROSS JOIN** departement

nomemp	nomdep
Karim	Informatique
Karim	RH
Karim	Finance
Karim	Marketing
Ali	Informatique
Ali	RH
...	...

Exprimer la jointure ?

La jointure avec qualification peut être exprimée en SQL de plusieurs manières

- en utilisant la restriction (la sélection) du produit cartésien :
SELECT *
FROM liste de noms de relations
WHERE condition de jointure
- En utilisant l'imbrication des requêtes.

Remarque :

les opérateurs de jointure, de sélection et de projection peuvent être combinés et effectués à l'intérieur d'une même clause SELECT.

Jointure: ancienne notation

- La jointure peut aussi être traduite par la clause WHERE :

SELECT nomemp, nomdep

FROM employe , departement

WHERE employe.coddep =departement.coddep

- Cette façon de faire est encore très souvent utilisée, même avec les SGBD qui supportent la syntaxe SQL2.

Exprimer la jointure ?

- **Donner le nom de département qui a un employé dont le nom 'Ali'.**

```
SELECT d.Nomdep  
FROM departement d, employe e  
WHERE d.coddep = e.coddep  
AND e.nomemp = 'Ali';
```

Jointure (norme SQL2):

Liste des noms d'employés avec le nom du département où ils travaillent :

```
SELECT nomemp, nomdep  
FROM employe JOIN departement ON  
employe.coddep = departement.coddep;
```

Jointure (norme SQL2):

Liste des noms d'employés avec le nom du département où ils travaillent :

nomemp	nomdep
Karim	RH
Ali	Informatique
Sarah	Finance
Yasmine	RH
Omar	RH

Jointure naturelle

Syntaxe:

```
SELECT Nomemp, nomdep  
FROM employe NATURAL JOIN departement
```

REMARQUES:

- on n'a pas besoin d'indiquer les colonnes de jointure car la clause **natural join** joint les deux tables sur toutes les colonnes qui ont le même nom dans les deux tables.
- Si on utilise une jointure naturelle, il est interdit de préfixer une colonne utilisée pour la jointure par un nom de table. La requête suivante provoque une erreur :

```
SELECT Nomemp, nomdep, departement.coddep  
FROM employe NATURAL JOIN departement
```

Jointure naturelle (suite)

- Il faut écrire :
SELECT Nomemp, nomdep, coddep
FROM employe **NATURAL JOIN** departement
- Comment obtenir la jointure naturelle sur une partie seulement des colonnes qui ont le même nom?
il faut utiliser la clause **__join using__** (s'il y a plusieurs colonnes, le séparateur de colonnes est la virgule).
- La requête suivante est équivalente à la précédente :
SELECT nomemp, nomdep
FROM employe **JOIN** departement **USING** coddep

Left Join

LEFT JOIN : Conserver tous les enregistrements de la table de gauche

Définition :

Un LEFT JOIN (ou LEFT OUTER JOIN) récupère **toutes les lignes** de la table de gauche et les associe avec celles de la table de droite.

- Si une correspondance est trouvée, les valeurs de la table droite sont affichées.
- S'il n'y a pas de correspondance, les valeurs de la table droite seront NULL.

Exemple:

Quels sont les employés et les départements auxquels ils appartiennent, y compris ceux qui ne sont affectés à aucun département ?

```
SELECT employe.codemp, employe.nomemp, employe.coddep,  
departement.nomdep
```

```
FROM employe
```

```
LEFT JOIN departement ON employe.coddep = departement.coddep;
```

Left Join

SELECT employe.codemp, employe.nomemp, employe.coddep, departement.nomdep
FROM employe
LEFT JOIN departement **ON** employe.coddep = departement.coddep;

codemp	nomemp	datrecrut	codpos te	Salai re	Prime	Superieur	coddep
101	karim	2020-09-25	P002	4500	400	104	D20
102	Ali	2019-06-12	P001	6000	600	102	D10
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20
106	Hiba	2025-03-24	P006	3500	NULL	102	NULL

coddep	Nomdep	codrespon
D10	Informatique	102
D20	RH	104
D30	Finance	103
D40	Marketing	NULL

Left Join

Résultat:

```
SELECT employe.codemp, employe.nomemp, employe.coddep,  
departement.nomdep  
FROM employe  
LEFT JOIN departement ON employe.coddep = departement.coddep;
```

codemp	Nomemp	Coddep	Nomdep
101	karim	D20	RH
102	Ali	D10	Informatique
103	Sarah	D30	Finance
104	Yasmine	D20	RH
105	Omar	D20	RH
106	Hiba	NULL	NULL

Right Join

RIGHT JOIN : Conserver tous les enregistrements de la table de droite

Définition :

Un RIGHT JOIN (ou RIGHT OUTER JOIN) récupère **toutes les lignes** de la table de droite et les associe avec celles de la table de gauche.

- Si une correspondance est trouvée, les valeurs de la table gauche sont affichées.
- S'il n'y a pas de correspondance, les valeurs de la table gauche seront NULL.

Exemple:

Lister tous les départements et les employés qui y sont affectés, y compris les départements qui n'ont aucun employé associé

```
SELECT employe.codemp, employe.nomemp, departement.nomdep FROM  
employe RIGHT JOIN departement ON employe.coddep =  
departement.coddep;
```

Right Join

Résultat:

```
SELECT e.codemp, e.nomemp, e.coddep, d.nomdep  
FROM Employe e RIGHT JOIN Departement d  
ON e.coddep = d.coddep;
```

codemp	nomemp	codedep	nomdep
101	Karim	D20	RH
102	Ali	D10	Informatique
103	Sarah	D30	Finance
104	Yasmine	D20	RH
105	Omar	D20	RH
NULL	NULL	D40	Marketing

Jointure d'une table avec elle-même

- Il peut être utile de rassembler des informations provenant d'une ligne d'une table avec des informations venant d'une autre ligne de la même table.
- Dans ce cas il faut **renommer au moins l'une des deux tables** en lui donnant un **synonyme** , afin de pouvoir préfixer sans ambiguïté chaque nom de colonne.

Jointure d'une table avec elle-même: Auto-jointure (Self JOIN)

Autojointure : Faire une jointure sur la même table

Définition :

L'auto-jointure consiste à faire une jointure d'une table avec elle-même. Cela est utile pour comparer des lignes d'une même table.

Cas d'utilisation : Trouver les employés et leurs supérieurs hiérarchiques.

Jointure d'une table avec elle-même

- Lister les employés qui ont un supérieur, en indiquant pour chacun le nom de son supérieur :

```
SELECT employe.nomemp as employe, supe.nomemp as superieur  
FROM   employe join employe supe  
       on employe.superieur = supe.codemp;
```

ou

```
SELECT employe.nomemp employe, supe.nomemp superieur  
FROM   employe, employe supe  
WHERE  employe.superieur = supe.codemp;
```

Jointure d'une table avec elle-même

- Lister les employés qui ont un supérieur, en indiquant pour chacun le nom de son supérieur :

codemp	nomemp	datrecrut	codpos te	Salai re	Prime	Superieur	coddep
101	karim	2020-09-25	P002	4500	400	104	D20
102	Ali	2019-06-12	P001	6000	600	102	D10
103	Sarah	2018-11-03	P003	3800	350	102	D30
104	Yasmine	2020-04-18	P004	5200	200	102	D20
105	Omar	2017-07-30	P005	5900	NULL	NULL	D20
106	Hiba	2025-03-24	P006	3500	NULL	102	NULL

Jointure d'une table avec elle-même

- Lister les employés qui ont un supérieur, en indiquant pour chacun le nom de son supérieur :

```
SELECT employe.nomemp AS employe, supe.nomemp AS superieur
FROM   employe join employe supe
       on employe.superieur = supe.codemp
WHERE  employe.codemp <> supe.codemp;
```

ou

```
SELECT employe.nomemp AS employe, supe.nomemp AS superieur
FROM   employe, employe supe
WHERE  employe.superieur = supe.codemp
AND    employe.codemp <> supe.codemp;
```

Jointure d'une table avec elle-même

- Lister les employés qui ont un supérieur, en indiquant pour chacun le nom de son supérieur :

Employe	Superieur
Karim	Yasmine
Sarah	Ali
Yasmine	Ali
Hiba	Ali

Jointure d'une table avec elle-même

- Afficher la liste de tous les employés ainsi que le nom de leur supérieur hiérarchique, s'il existe. Inclure également les employés sans supérieur.

```
SELECT employe.nomemp AS employe, supe.nomemp AS superieur
FROM   employe LEFT join employe supe
      on employe.superieur = supe.codemp
WHERE  employe.codemp <> supe.codemp;
```

Employe	Superieur
Karim	Yasmine
Sarah	Ali
Yasmine	Ali
Omar	NULL
Hiba	Ali

Jointure d'une table avec elle-même

- Afficher le code de département, le nom de chaque employé et le nom de ses collègues qui travaillent dans le même département.

```
SELECT e1.coddep, e1.nomemp AS employe, e2.nomemp AS colleague  
FROM Employe e1  
JOIN Employe e2 ON e1.coddep = e2.coddep  
WHERE e1.codemp <> e2.codemp;
```

coddep	employe	colleague
D10	Ali	Sarah
D20	Karim	Yasmine
D20	Karim	Omar
D20	Yasmine	Karim
D20	Yasmine	Omar
D20	Omar	Karim
D20	Omar	Yasmine

Jointure non-équi

- Les jointures autres que les équi-jointures (**inéqui-jointures**) peuvent être représentées :
- En remplaçant dans la clause **ON** ou la clause **WHERE** le signe **=** par un des opérateurs de comparaison (**<**, **<=**, **>**, **>=**, **!=**), ou encore **between** et **in**.

Jointure non-équi

On considère la table Salgrade suivante:

Grade	Salmin	Salmax
A	2000	3000
B	3001	4000
C	4001	5000
D	5001	6000

Créez une requête pour afficher le nom, le nom de département, le salaire et le grade de tous les employés.

Jointure non-équi

```
SELECT e.nomemp, d.nomdep, e.Salaire, s.grade  
FROM employe e  
JOIN departement d ON e.coddep = d.coddep  
JOIN salgrade s ON e.Salaire BETWEEN s.salmin AND s.salmax;
```

Nomemp	Nomdep	Salaire	Grade
Karim	RH	4500	C
Ali	Informatique	6000	D
Sarah	Finance	3800	B
Yasmine	RH	5200	C
Omar	RH	5900	D
Hiba	NULL	3500	B

Fonctions de groupes

- Les fonctions de groupe agissent sur des groupes de lignes et donnent un résultat par groupe.
- Les fonctions de groupe les plus utilisées : SUM, AVG, COUNT, MAX et MIN.
- Les fonctions de groupes sont souvent utilisées avec la clause GROUP BY pour calculer une valeur agrégée pour chaque groupe, par exemple la valeur moyenne du groupe ou la somme des valeurs dans chaque groupe.

Fonctions de groupes

Chaque fonction accepte un argument. La table suivante présente les différentes options de syntaxe possibles:

Fonction	Rôle
AVG([DISTINCT ALL]col)	Moyenne: Valeur moyenne de n, en ignorant les valeurs NULL
MIN([DISTINCT ALL]col)	Plus petite valeur : Valeur minimale de col, en ignorant les valeurs NULL
SUM([DISTINCT ALL]n)	Somme des valeurs de n, en ignorant les valeurs NULL
MAX([DISTINCT ALL]col)	Plus grande valeur: Valeur maximale de col, en ignorant les valeurs NULL
VARIANCE([DISTINCT ALL]n)	Variance de n, en ignorant les valeurs NULL
Count(*)	Nombre de lignes
Count(DISTINCT col)	Nombre de valeurs non nulles différentes
Count(col)	Nombre de valeurs non nulle de la colonne col
STDDEV([DISTINCT ALL]n)	Écart type(dérivation standard), en ignorant les valeurs NULL

Fonctions de groupes

Exemple 1:

- Nombre total des employés en service?

SELECT COUNT(*)

FROM employe;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



Count(*)

5

Fonctions de groupes

La fonction COUNT se présente sous deux formats :

- COUNT(*)
- COUNT(expr)

COUNT(*) ramène le nombre de lignes d'une table, y compris les lignes en double et celles contenant des valeurs NULL.

A l'opposé, COUNT(expr) ramène le nombre de lignes pour lesquelles la colonne identifiée par expr est non NULL.

Fonctions de groupes

Exemple 2:

- Nombre total des employés ayant une prime attribuée

SELECT COUNT(Prime)

FROM employe;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



Count(*)

4

Fonctions de groupes

Exemple 3:

- Valeur globale des primes des employés du département 20?

SELECT **SUM**(prime)

FROM employe

WHERE coddep = D20;

codemp	nomemp	datrec rut	codpost e	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



SUM(prime)

600

Fonctions de groupes

Exemple 4:

Afficher le salaire le plus élevé, le salaire le plus bas et le salaire moyen de l'ensemble des employés de la table employe

```
SELECT MAX(Salaire) AS salaire_max, MIN(Salaire) AS salaire_min,  
AVG(Salaire) AS salaire_moyen FROM employe;
```

codemp	nomemp	datrecr ut	codpost e	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



salaire_max	salaire_min	salaire_moyen
6000	3800	5080

Fonctions de groupes

les fonctions MAX et MIN s'utilisent pour tous les types de données.

SELECT MIN(datrecrut), MAX(datrecrut)

FROM employe;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



MIN(datrecrut)	MAX(datrecrut)
2017-07-30	2020-09-25

Remarque : les fonctions AVG, SUM, VARIANCE et STDDEV n'acceptent que des données de type numérique

Fonctions de groupes

Exemple 5:

Afficher la moyenne des primes attribuées aux employés.

```
SELECT AVG(Prime)
FROM employe;
```

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



AVG(Prime)

387.5

Fonctions de groupes

- Les fonctions de groupes peuvent apparaître dans le « **Select** » ou le « **Having** »
- Les valeurs **NULL** sont ignorées par les fonctions de groupe.
 - **SUM(col)** est la somme des valeurs **différentes** de **NULL** de la colonne 'col'.
 - **AVG** : (**somme** des valeurs **non NULL**)/ (nombre de valeurs **non NULL**).
- Toutes les fonctions de groupe, à l'exception de COUNT(*), ignorent les valeurs NULL. Pour substituer une valeur à une valeur NULL, utilisez la fonction COALESCE()

Fonctions de groupes

SELECT AVG(COALESCE(Prime, 0)) **AS** moyenne_prime
FROM employe;

codemp	nomemp	datrecrut	codposte	Salaire	Prime	Superieur	coddep
101	karim	2020-09-25	P001	4500	400	104	D20
102	Ali	2019-06-12	P002	6000	600	102	D10
103	Sarah	2018-11-03	P004	3800	350	102	D30
104	Yasmine	2020-04-18	P001	5200	200	102	D20
105	Omar	2017-07-30	P002	5900	NULL	NULL	D20



moyenne_prime

310

Fonctions de groupes

Création de groupe de données

	codemp	nomemp	datrecr ut	codpost e	Salaire	Prime	Superieur	coddep
10	102	Ali	2019-06-12	P001	6000	600	102	D10
10	101	karim	2020-09-25	P002	4500	400	104	D20
10	104	Yasmine	2020-04-18	P004	5200	200	102	D20
10	105	Omar	2017-07-30	P001	5900	NULL	NULL	D20
10	103	Sarah	2018-11-03	P002	3800	350	102	D30



Coddep	AVG(salaire)
D10	6000
D20	5200
D30	3800

Clause GROUP BY syntaxe

SELECT <colonnes, fonctions de groupe>
FROM < expression de tables>
[WHERE < condition de recherche>**]**
GROUP BY <expressions group by>
[HAVING < condition de groupe >**]**

Clause GROUP BY exécution

Exécuter la clause 'FROM <expression de tables>

Si le 'WHERE' existe, sélectionner les lignes qui vérifient le prédicat

Exécuter le 'GROUP BY' qui répartit les lignes dans des groupes où les colonnes du group by ont toutes la même valeur. Les 'NULL' sont traités comme s'ils étaient égaux entre eux, et forment donc leur propre groupe.

Appliquer les fonctions de calcul spécifiées dans la clause SELECT sur les groupes générés. Chaque groupe est alors réduit à une seule ligne dans une nouvelle table de résultats.

S'il y a une clause 'HAVING', elle s'applique à tous les groupes. Les groupes pour lesquels le test donne 'true' sont retenus, ceux qui donnent 'false' ou 'unknown' sont supprimés.

Clause GROUP BY

Rôle:

Subdiviser la table en groupes, chaque groupe contient l'ensemble des lignes ayant la même valeur

GROUP BY exp1, exp2,... :

groupe en une seule ligne toutes les lignes pour lesquelles exp1, exp2,... ont la même valeur.

Group By se place :

- juste après la clause **WHERE**
- après la clause **FROM** si la clause **WHERE** n'existe pas.

Des lignes peuvent être éliminées avant que le groupe ne soit formé grâce à la clause **WHERE**.

Clause GROUP BY: exemples

- Afficher le nombre d'employés de chaque département
SELECT coddep, **COUNT**(*)
FROM employe
GROUP BY coddep;

coddep	Count(*)
D10	1
D20	3
D30	1

Clause GROUP BY: exemples

- La colonne citée en GROUP BY ne doit pas nécessairement figurer dans la liste SELECT.

```
SELECT COUNT(*)  
FROM employe  
GROUP BY coddep;
```

Count(*)
1
3
1

Clause GROUP BY: exemples

- Afficher, pour chaque département, le nombre d'employés ayant un salaire supérieur à 4000.

SELECT coddep, **COUNT**(*)

FROM employe

WHERE Salaire > 4000

GROUP BY coddep;

Coddep	Count(*)
D10	1
D20	3

Clause GROUP BY:

Exemple: Regroupement sur plusieurs colonnes

Afficher le nombre d'employés occupant le même poste dans chaque département:

```
SELECT coddep, codposte, COUNT(*)  
FROM employe  
GROUP BY coddep, codposte;
```

coddep	codposte	Count(*)
D10	P002	1
D20	P001	2
D20	P002	1
D30	P004	1

Clause GROUP BY : RESTRICTION

- Une expression d'un **SELECT** avec clause **GROUP BY** doit correspondre à une caractéristique de groupe.
- Le **SELECT** contient :
 - soit une fonction de groupe,
 - soit une expression figurant dans le **GROUP BY**.
- Dans une requête avec **GROUP BY**, **toutes les colonnes** présentes dans la clause **SELECT** doivent **soit** :
 1. Appartenir à la clause **GROUP BY**,
OU
 2. Être utilisées à l'intérieur d'une **fonction d'agrégation** (SUM, COUNT, AVG, MIN, MAX...).

Clause GROUP BY : RESTRICTION

- Avec la clause **WHERE**, vous pouvez exclure des lignes avant de créer des groupes.
- Vous devez inclure les "column" de la liste **SELECT** dans la clause **GROUP BY**.
- Vous ne pouvez pas utiliser l'alias de colonne dans la clause **GROUP BY**.
- Par défaut, les lignes sont triées dans l'ordre croissant des colonnes incluses dans la liste **GROUP BY**.
- Vous pouvez changer cet ordre en utilisant la clause **ORDER BY**.

Clause GROUP BY : RESTRICTION

- Exemple de requête erronée :

(nomdep ne figure pas dans le **GROUP BY**)
SELECT nomdep , **SUM**(salaire)
FROM employe **NATURAL JOIN** departement
GROUP BY coddep

La clause GROUP BY doit inclure toutes les colonnes de la liste SELECT qui ne figurent pas dans des fonctions de groupe

Clause GROUP BY: Restriction

Il faut, soit se contenter du numéro de département au lieu du nom :

```
SELECT coddep, SUM(salaire)
FROM employe NATURAL JOIN departement
GROUP BY coddep;
ou bien:
```

```
SELECT nomdep, SUM(salaire)
FROM employe NATURAL JOIN departement
GROUP BY nomdep;
ou bien:
```

```
SELECT coddep, nomdep, SUM(salaire)
FROM employe NATURAL JOIN departement
GROUP BY coddep, nomdep;
```

Clause GROUP BY: Restriction

```
SELECT coddep, nomdep, SUM(salaire)
FROM employe NATURAL JOIN departement
GROUP BY coddep, nomdep;
```

coddep	nomdep	SUM(salaire)
D10	Informatique	6000
D20	Marketing	15600
D30	Ressources Humaines	3800

Exclusion de Groupes : Clause HAVING

- Prédicat qui sert à préciser quels groupes doivent être sélectionnés.
- Elle se place après la clause **GROUP BY**.
- **HAVING** suit la même syntaxe que le **WHERE**.
- Il ne peut porter que sur des fonctions de groupe ou des expressions figurant dans la clause **GROUP BY**.

Exclusion de Groupes : Clause HAVING

- Utilisez la clause **HAVING** pour restreindre les groupes:
 - Les lignes sont regroupées.
 - La fonction de groupe est appliquée.
 - Les groupes qui correspondent à la clause **HAVING** sont affichés.

Clause HAVING

Exemple

- Afficher les départements où le total des salaires dépasse 10 000:

```
SELECT coddep, SUM(Salaire) AS total_salaire  
FROM employe  
GROUP BY coddep  
HAVING SUM(Salaire) > 10000;
```

coddep	total_salaire
D20	15600

Clause HAVING

Exemple

- Afficher les départements dans lesquels le salaire le plus élevé est supérieur à 5900:

```
SELECT coddep, MAX(salaire)  
FROM employe  
GROUP BY coddep  
HAVING MAX(salaire) > 5900;
```

coddep	MAX(salaire)
D10	6000

Clause HAVING

Exemple

- Afficher, pour chaque département, la moyenne des salaires uniquement si le salaire maximum de ce département est supérieur à 4000.

SELECT coddep, AVG(salaire)

FROM employe

GROUP BY coddep

HAVING MAX(salaire) > 4000;

coddep	AVG(salaire)
D10	6000
D20	5200

Opérateurs ensemblistes

- Les opérateurs ensemblistes permettent de combiner les résultats de plusieurs requêtes SQL.
 - Les opérateurs ensemblistes permettent de "joindre" des tables verticalement c'est-à-dire de combiner dans un résultat unique des lignes provenant de deux interrogations.
 - Les lignes peuvent venir de tables différentes mais après projection on doit obtenir des tables ayant même schéma de relation.
- Similaires aux opérations ensemblistes mathématiques : **UNION (UNION ALL), INTERSECT, EXCEPT.**

Opérateurs ensemblistes

Dans une requête utilisant des opérateurs ensemblistes :

- Tous les SELECT doivent avoir le même nombre de colonnes sélectionnées, et leur types doivent être un à un identiques. Les conversions éventuelles doivent être faites à l'intérieur du SELECT à l'aide des fonctions de conversion.
- Les doubles sont éliminés (**DISTINCT** implicite).
- Les noms de colonnes (titres) sont ceux du premier SELECT.
- Dans une requête on ne peut trouver qu'un seul ORDER BY. S'il est présent, il doit être mis dans le dernier SELECT et il ne peut faire référence qu'aux numéros des colonnes et non pas à leurs noms (car les noms peuvent être différents dans chacune des interrogations).

Opérateurs ensemblistes

Opérateur **UNION**

- **fusionner** deux sélections de tables pour obtenir un ensemble de lignes égal à la réunion des lignes des deux sélections.
- Les lignes communes n'apparaîtront qu'une fois.

Syntaxe: (les deux tables ont le **même schéma**)

SELECT colonne1, colonne2 **FROM** table1 **WHERE** ...

UNION

SELECT colonne1, colonne2 **FROM** table2 **WHERE** ...

UNION ALL:

- Fusionne deux jeux de résultats, en gardant les doublons.
- Plus rapide que UNION car il ne gère pas les doublons.

Opérateurs ensemblistes

Opérateur **UNION**

Exemple:

Récupérer les codes des employés et des responsables de département, sans doublons.

SELECT codemp FROM Employe

UNION

SELECT codrespon FROM Departement;

codemp
101
102
103
104
105
NULL

Opérateurs ensemblistes

Opérateur UNION

codemp	nomemp	datrecr	codpost	Salaire	Prime	Superieur	coddep	
101	karim	codemp		1	4500	400	codrespon	
		101					102	
102	Ali	102		2	5000	600	104	
		103					103	
103	Sarah	104		4	3500	350	NULL	
		105						
104	Yasmine	04-18			codemp			
101	coddep		Nomdep	101		codrespon		
	D10		Informatiq	102		102		
	D20		RH	103		104		
	D30		Finance	104		103		
	D40		Marketing	105		NULL		
				NULL				

Opérateurs ensemblistes

Opérateur **UNION ALL**

Cet opérateur permet de ramener toutes les lignes issues de plusieurs requêtes. Contrairement à l'opérateur UNION, les doublons ne sont pas éliminés et le résultat n'est pas trié par défaut. Il n'est pas possible d'utiliser le mot-clé DISTINCT

Exemple:

Récupérer les codes des employés et des responsables de département.

```
SELECT codemp FROM Employe
```

```
UNION ALL
```

```
SELECT codrespon FROM Departement;
```

codemp
101
102
102
103
103
104
104
105
NULL

Opérateurs ensemblistes

Opérateur **UNION ALL**

codemp	nomen	codemp	post	Salaire	Prin	codrespon		
101	karim	101	1	500	400	102		
		102				104		
102	Ali	103	2	6000	600	103		
		104				NULL		
103	Sarah	105	4	101			102	D30
104	Yasmine	2020-04-18	P001	102			102	D20
				102				
1	coddep		Nomdep	103	odrespon			
	D10		Informati	103	02			
	D20		RH	104	04			
	D30		Finance	104	03			
	D40		Marketing	105	NULL			
				NULL				

Opérateurs ensemblistes

Opérateur **INTERSECT**

- Permet d'obtenir l'ensemble des lignes **communes** à deux interrogations.

Syntaxe: (les deux tables ont le **même schéma**)

```
SELECT colonne1, colonne2 FROM table1 WHERE .....
```

```
INTERSECT
```

```
SELECT colonne1, colonne2 FROM table2 WHERE.....
```

Opérateurs ensemblistes

Opérateur **INTERSECT**

Exemple: Liste des numéros des employés qui sont aussi responsables de département.

```
SELECT codemp FROM Employe  
INTERSECT
```

```
SELECT codrespon FROM Departement;
```

codemp
102
103
104

Opérateurs ensemblistes

Opérateur INTERSECT

codemp	nomemp	datrecr	codpost	Salaire	Prime	Superieur	coddep
101	karim	01-18	1	4500	400		
102	Ali	02-18	2	6000	600		
103	Sarah	04-18	4	3800	350		
104	Yasmine	04-18	1	5200	200	102	D20
coddep	Nomdep	codemp	codrespon				
D10	Informatique	102	102				
D20	RH	103	104				
D30	Finance	104	103				
D40	Marketing		NULL				

Opérateurs ensemblistes

Opérateur **EXCEPT**

- permet d'ôter d'une sélection les lignes obtenues dans une deuxième sélection.

Syntaxe: (les deux tables ont le **même schéma**)

SELECT colonne1, colonne2 **FROM** table1 **WHERE** ...

EXCEPT

SELECT colonne1, colonne2 **FROM** table2 **WHERE**...

Opérateurs ensemblistes

Opérateur **EXCEPT**

Exemple:

Lister les employés qui ne sont pas responsables de département.

```
SELECT codemp FROM Employe  
EXCEPT  
SELECT codrespon FROM Departement;
```

codemp
101
105

Opérateurs ensemblistes

Opérateur EXCEPT

codemp	nomemp	datrecr	codpost	Salaire	Prime	Superieur	coddep
101	codemp	2020-09-25	P001	4500	codrespon		D20
102	101	2019-06-12		6000	102		D10
103	102	2018-11-03	P004	3800	104		D30
104	103				103		
105	104				NULL		
104	105	2020-04-18	codemp	500	200	102	D20
105	Omar	2017-07-30	101	500	NULL	NULL	D20

coddep	Nomdep	codrespon
D10	Informatique	102
D20	RH	104
D30	Finance	103
D40	Marketing	NULL

Sous-interrogation

"Qui a un salaire inférieur à celui de Ali ?«

C'est exactement ce que fait une **sous-requête** : Elle nous permet de **découper un problème en deux parties** :

- Trouver une information intermédiaire (le salaire d'Ali),
- Utiliser cette information dans une requête plus large.

Requête principale

"Quel employé a un salaire inférieur à celui de Ali ? "



Sous-interrogation

"Quel est le salaire de Ali ?"



Sous-interrogation

- Une caractéristique puissante de SQL est la possibilité qu'un prédicat employé dans une clause **WHERE** comporte un **SELECT** emboîté.
- Une sous-requête est une requête incluse dans une autre requête (principale), i.e., la requête principale utilise le résultat de la sous-requête
- La requête interne, ou sous-interrogation, ramène une valeur utilisée par la requête externe, ou principale. Utiliser une sous-interrogation revient à exécuter deux requêtes successives en utilisant le résultat de la première comme valeur de recherche de la seconde.
- Les sous-requêtes sont utilisables dans les clauses :
 - select (à condition que pour chaque ligne sélectionnée par la requête principale, on ne sélectionne qu'une ligne dans la sous-requête)
 - from (à condition de renommer le résultat)
 - where et having

Sous-interrogation

Principe:

Lorsque dans un prédicat un des deux arguments n'est pas connu, on utilise les sous-interrogations.

SELECT...

FROM ...

WHERE variable Op ?

Le ? n'est pas connu, il sera le résultat d'une sous-requête.

Règles d'exécution:

C'est la sous-requête de niveau le plus bas qui est évaluée en premier, puis la requête de niveau immédiatement supérieur,...

Sous-interrogation

Syntaxe:

SELECT select_list

FROM table

WHERE expr **operator**

(SELECT select_list
FROM table);

- La sous-interrogation(requête interne) est exécutée une fois avant la requête principale.
- Le résultat de la sous-interrogation est utilisé par la requête principale (externe).

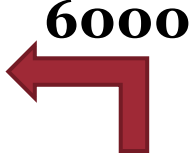
Operator est un opérateur de comparaison tel que >, = ou IN.

Remarque : les opérateurs de comparaison se classent en deux catégories : les opérateurs mono ligne (>, =, >=, <>, <=) et les opérateurs multi-ligne (IN, ANY, ALL).

Sous-interrogation

Résultat de l'exemple:

```
SELECT nomemp  
FROM employe  
WHERE salaire < (SELECT salaire  
                  FROM employe  
                  WHERE nomemp='ALI');
```



nomemp
Karim
Sarah
Yasmine
Omar

la requête interne recherche quel est le salaire de l'employé dont le nom est ALI.

La requête externe prend le résultat de la requête interne et l'utilise pour afficher tous les employés qui gagnent moins que cette somme.

Sous-interrogation

Principes d'utilisation des sous-interrogations:

- Placez les sous-interrogations entre parenthèses.
- Placez les sous-interrogations à droite de l'opérateur de comparaison.
- N'ajoutez jamais de clause ORDER BY à une sous-interrogation.
- Utilisez les opérateurs mono-ligne avec les sous-interrogations mono-ligne.
- Utilisez les opérateurs multi-ligne avec les sous-interrogations multi-ligne.

Sous-interrogation

Types de sous-interrogations:

- Sous-interrogation mono-ligne
- Sous-interrogation multi-ligne
- Sous-interrogation multi-colonne
- Sous-interrogation synchronisées

Sous-interrogation Mono-ligne

- Ne ramènent qu'une seule ligne
- Utilisent des opérateurs de comparaison mono-ligne

Opérateur	Signification
=	Egal à
>	Supérieur à
>=	Supérieur ou égal à
<	Inférieur à
<=	Inférieur ou égal à
<>	Différent de

Sous-interrogation Mono-ligne

Exemple:

Quels sont les employés ayant le même département que l'employé de code 105:

```
SELECT nomemp  
FROM employe  
WHERE coddep = (SELECT coddep  
                FROM employe  
                WHERE codemp = 105)
```

And codemp != 105

nomemp
karim
Yasmine

Sous-interrogation Mono-ligne

Exemple:

Quels sont les employés ayant le même département que l'employé de code 105 et ayant un salaire inférieur à celui de Ali :

```

SELECT nomemp
FROM employe
WHERE coddep = (SELECT coddep
                FROM employe
                WHERE codemp = 105)
                6000
And salaire < (SELECT salaire
               FROM employe
               WHERE nomemp='ALI');
  
```

nomemp
karim
Yasmine

Sous-interrogation Mono-ligne

Exemple:

Donner le nom et le salaire de l'employé ayant le salaire le plus bas:

```
SELECT nomemp, salaire  
FROM employe  
WHERE salaire = (SELECT Min(salaire)  
                FROM employe);
```

nomemp	Salaire
Sarah	3800

Sous-interrogation Mono-ligne

Exemple:

Afficher les départements dont le salaire minimum est supérieur au salaire minimum du département D20:

```
SELECT coddep, MIN(salaire)
FROM employe
GROUP BY coddep
HAVING MIN(salaire) > (SELECT Min(salaire)
FROM employe
WHERE coddep='D20');
```

coddep	Min(Salaire)
D10	6000

Sous-interrogation Mono-ligne

Erreur Liée aux Sous-Interrogations ✕

```
SELECT nomemp, salaire  
FROM employe  
WHERE salaire = (SELECT Min(salaire)  
                  FROM employe  
                  GROUP BY coddep);
```

Sous-interrogation Multi-lignes

- Une **sous-interrogation** peut ramener **plusieurs** lignes :
- Les opérateurs permettant de **comparer** une valeur à un **ensemble** de valeurs sont :
 - l'opérateur **IN**
 - **ANY** ou **ALL** à la suite des **opérateurs** de comparaison: (=, !=, <, >, <=, >=)
- Forme de la sous-requête:
 - WHERE** exp **op ANY** (SELECT ...)
 - WHERE** exp **op ALL** (SELECT ...)
 - WHERE** exp **IN** (SELECT ...)
 - WHERE** exp **NOT IN** (SELECT ...)

Sous-interrogation Multi-lignes : les opérateurs **ANY** et **ALL**

WHERE exp **op ANY** (SELECT ...)

ANY : la comparaison sera vraie si elle est vraie pour au moins un élément de l'ensemble (elle est donc fausse si l'ensemble est vide).

WHERE exp **op ALL** (SELECT ...)

ALL : la comparaison sera vraie si elle est vraie pour tous les éléments de l'ensemble.

Sous-interrogation Multi-lignes : les opérateurs **ANY** et **ALL**

Remarques:

- L'opérateur **IN** est équivalent à **= ANY**
- l'opérateur **NOT IN** est équivalent à **!= ALL**.
- **<ANY** signifie inférieur à au moins une des valeurs, donc inférieur au maximum.
- **>ANY** signifie supérieur à au moins une des valeurs, donc supérieur au minimum.
- **>ALL** signifie supérieur au maximum
- **<ALL** signifie inférieur au minimum

Sous-interrogation Multi-lignes : les opérateurs **ANY** et **ALL**

Exemple:

Quels sont les employés ayant le salaire le plus bas dans chaque département :

SELECT nomemp, salaire, coddep

FROM employe

WHERE salaire **IN**

(**SELECT** MIN(salaire) **FROM** employe
GROUP BY coddep);

nomemp	salaire	coddep
Karim	4500	D20
Ali	6000	D10
Sarah	3800	D30

Sous-interrogation Multi-lignes : les opérateurs ANY et ALL

Exemple:

Liste des employés gagnant plus que tous les employés du département 30 :

SELECT nomemp, salaire

FROM employe

WHERE salaire > **ALL**

(**SELECT** salaire **FROM** employe

WHERE coddep='D30');

nomemp	salaire
Karim	4500
Ali	6000
Yasmine	5200
Omar	5900

Sous-interrogation Multi-lignes : les opérateurs ANY et ALL

Exemple:

Afficher les employés dont le salaire est supérieur à tous les salaires des employés ayant le poste 'P001', à l'exclusion de 'P001':

```
SELECT codemp, nomemp, salaire, codposte
FROM employe
WHERE salaire > ALL
      (SELECT salaire FROM employe
        WHERE codposte='P001')
AND codposte <> 'P001';
```

codemp	nomemp	salaire	codposte
102	Ali	6000	P002
105	Omar	5900	P002

Sous-interrogation Multi-lignes

Exemple :

Afficher les employés dont le salaire est inférieur à celui de n'importe quel employé ayant le poste 'P001', et qui n'occupent pas eux-mêmes ce poste:

```
SELECT codemp, nomemp, salaire, codposte
FROM employe
WHERE salaire < ANY
  (SELECT salaire
   FROM employe
   WHERE codposte = 'P001')
AND codposte <> 'P001';
```

Codemp	Nomemp	Salaire	Codposte
103	Sarah	3800	P004

Sous-interrogation Multi-lignes

Exemple :

Liste des employés du département 10 ayant le même poste que quelqu'un du département VENTES :

```
SELECT nomemp, codposte
FROM employe
WHERE coddep = 10
AND codposte = ANY {or codposte in(.....)}
  (SELECT codposte
   FROM employe
   WHERE coddep =
     (SELECT coddep
      FROM departement
      WHERE nomdep = 'VENTES'))
```

Sous-interrogation Multi-colonnes

- Une **sous-interrogation** peut renvoyer **plusieurs** colonnes :
→ utile pour **comparer des couples ou des groupes de colonnes**
- Syntaxe:

```
SELECT    column, column, ...  
FROM      table  
WHERE      (column, column, ...) IN  
            (SELECT column, column, ...  
             FROM   table  
             WHERE  condition);
```

Sous-interrogation Multi-colonnes

Exemple :

afficher le nom, le numéro de département et le salaire de tout employé dont le numéro de département et le salaire sont tous les deux équivalents au numéro de département et au salaire de n'importe quel employé touchant une prime :

```
SELECT nomemp, coddep,salaire
FROM employe
WHERE (salaire,coddep) IN (SELECT e.salaire,e.coddep
                           FROM employe e
                           WHERE e.prime IS NOT NULL);
```

Sous-interrogation Multi-colonnes

Exemple :

```
SELECT nomemp, coddep,salaire
FROM employe
WHERE (salaire,coddep) IN (SELECT(salaire,coddep)
                           FROM employe
                           WHERE prime IS NOT NULL);
```

nomemp	coddep	salaire
Karim	D20	4500
Ali	D10	6000
Sarah	D30	3800
Yasmine	D20	5200

Sous-interrogation Multi-colonnes

Exemple :

Affichez le nom de l'employé, le nom du département et le salaire de tout employé dont le salaire et la prime sont tous les deux équivalents au salaire et à la prime de n'importe quel employé du département RH:

```
SELECT e.nomemp, d.nomdep, e.salaire
FROM employe e JOIN departement d ON e.coddep=d.coddep
WHERE (e.salaire, COALESCE(e.prime, 0)) IN (SELECT e1.salaire,
COALESCE(e1.prime, 0)
FROM employe e1 JOIN departement
d1 ON e1.coddep=d1.coddep
WHERE d1.nomdep='RH');
```


Sous-interrogation Multi-colonnes

Exemple :

```
SELECT e.nomemp, d.nomdep, e.salaire
FROM employe e JOIN departement d ON e.coddep=d.coddep
WHERE (e.salaire, COALESCE(e.prime, o)) IN (SELECT (e1.salaire,
COALESCE(e1.prime, o))
FROM employe e1 JOIN departement d1 ON
e1.coddep=d1.coddep
WHERE d1.nomdep='RH');
```

nomemp	Nomdep	salaire
Karim	RH	4500
Yasmine	RH	5200

Sous-Interrogations synchronisées

Principe:

Lorsque dans un prédicat un des deux arguments n'est pas connu, on utilise les sous-interrogations.

SELECT...

FROM ...

WHERE variable Op ?

Mais lorsque la valeur ? Est susceptible de varier pour chaque ligne, on utilise les sous-interrogations synchronisées.

Règles:

- Le prédicat de la sous-interrogation fait référence a une colonne de l'interrogation principale
- Si une table est présente dans les deux select, la renommer

Exécution:

- L'exécution de la sous-interrogation se fait pour chaque ligne de l'interrogation principale.

Sous-Interrogations synchronisées

- Une sous interrogation est synchronisée si elle manipule des colonnes d'une table du niveau supérieur.
- Une sous interrogation synchronisée est exécutée une fois pour chaque enregistrement extrait par la requête de niveau supérieur.
- La forme générale d'une sous interrogation synchronisée est la suivante. Les alias des tables sont utiles pour pouvoir manipuler des colonnes de tables de différents niveaux.

```
SELECT alias1.x ←  
FROM nomTable1 alias1  
WHERE [colonne(s)]opérateur(SELECT alias2.y...  
                                FROM nomTable2 alias2  
                                WHERE alias1.x opérateur alias2.y)  
[AND(conditionsTable1)] ;
```

Une sous interrogation synchronisée peut ramener une ou plusieurs lignes. Différents opérateurs peuvent être employés (=, >, <, >=, <=, EXISTS).

Sous-Interrogations synchronisées

Syntaxe avec EXISTS

```
SELECT *  
FROM Table alias1  
WHERE [NOT] EXISTS ( SELECT *  
                     FROM Table alias2  
                     WHERE alias2.idE = alias1.idE);
```

Sous-interrogation synchronisées

Exemple :

Afficher tous les employés qui ne sont pas responsables d'autres employés.

```
SELECT e1.nomemp  
FROM employe e1  
WHERE NOT EXISTS (  
    SELECT e2.codemp  
    FROM employe e2  
    WHERE e2.superieur = e1.codemp  
);
```

nomemp
Karim
Sarah
Omar

Sous-interrogation

Exercice: On considère les tables suivantes:

Client(codeclient, nom, prénom, ville, numtel)

Produit(codeproduit, description, catégorie, prix, quantité)

- 1- Afficher les clients qui habitent la même ville que le client Fadil
- 2- Afficher les clients qui habitent la même ville que les clients Fadil et Salim
- 3- Afficher les produits qui appartiennent a la même catégorie que le produit numéro 4
- 4- Afficher les produits les plus chers que les draps
- 5- Afficher les produits plus chers que les produits de type maison
- 6- Afficher les produits moins chers que les produits de type maison
- 7- Afficher les produits plus chers que l'un des produits de type maison
- 8- Afficher les produits moins chers que l'un des produits de type maison